

AI 기반 민원 분류 시스템

김훈종, 강진용, 이승윤, 이정륜

Safe Guard



AI·CLOUD·DATA 연계플랫폼 평생교육원

목차

○ 01 프로젝트 개요

시스템 설계 및 기술 02 ○

○ 03 핵심 기능 및 구현

문제 해결 방법 및 협업 관리 04 ○

○ 05 결론 및 향후 계획

01 프로젝트 개요

01

역할 소개

담당 역할

단계별 개발 일정
민원 접수의 문제점
구현 방향

👑 김훈종(팀장)

AWS 인프라 및 CI/CD 파이프라인 구축

Docker 기반 개발·배포 환경 구축

JWT 기반 회원 인증 및 계정 관리 로직 구현

민원 접수 트랜잭션 처리

강진용

권한(Role) 기반 민원 게시판 CRUD 및 처리 로직 구현

담당 기관별 민원 조회·답변·상태 변경 권한 제어

YOLO 기반 이미지 분류 모델 튜닝

민원 처리 단계(Status) 관리 및 상태 전환 API 설계

알람 실시간 기능 , 관리자 마이페이지 구현

이승윤

DB 설계 및 ERD 작성

YOLO 기반 이미지 분류 모델 튜닝

GIS 기반 민원 시각화 기능 구현

이미지 업로드 및 처리 로직 구현

마이페이지 기능 구현

이정륜

RAG 기반 AI 분류 시스템 · 벡터 DB(Milvus) 구축·운영

STT 음성 인식 처리

관리자 실시간 대시보드 시각화 구현

github 협업 템플릿 관리 (마일스톤/PR/ISSUE)

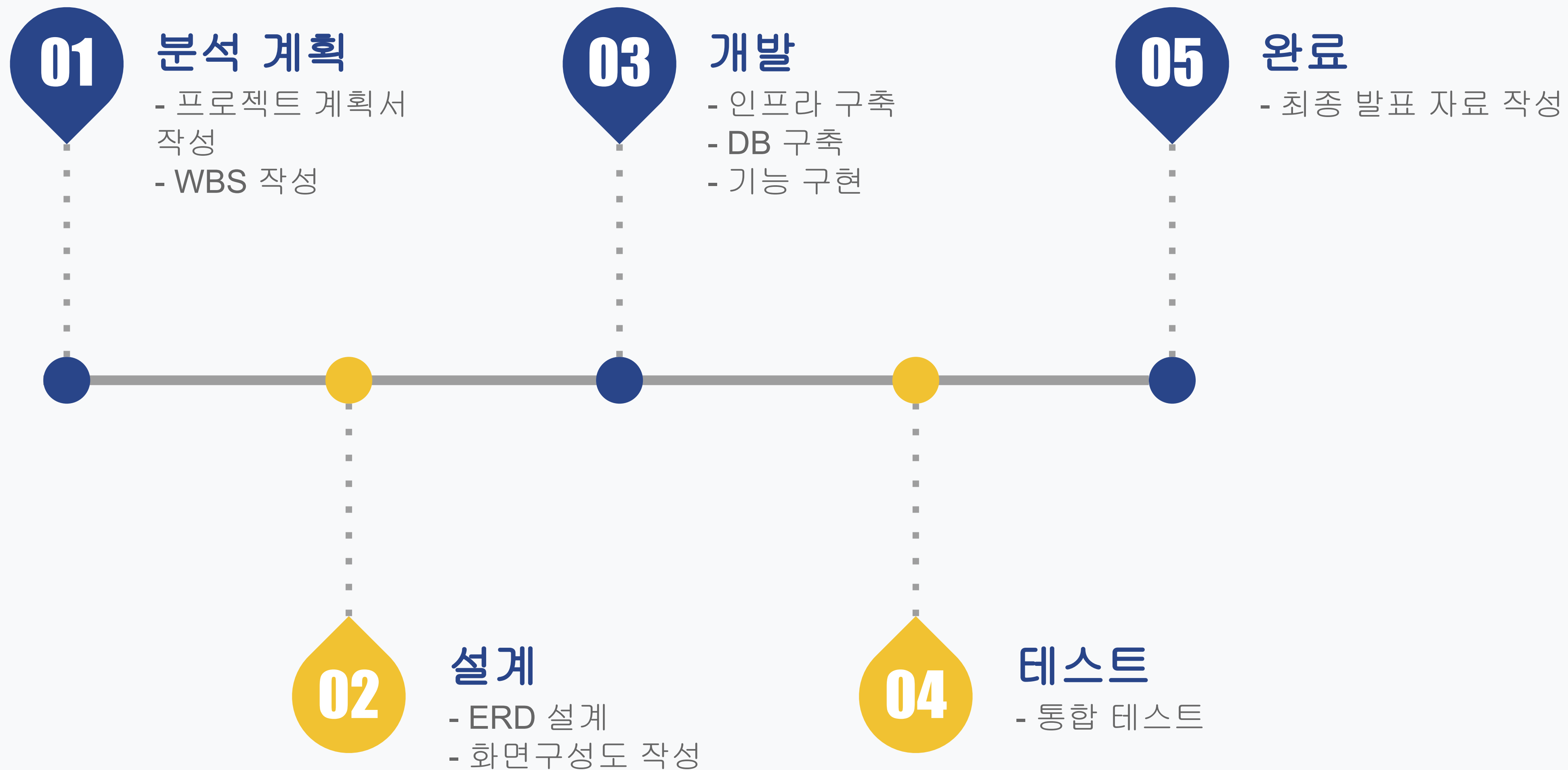
01

개발 기간

담당 역할

단계별 개발 일정

민원 접수의 문제점
구현 방향



01

개발 기간

담당 역할

단계별 개발 일정

민원 접수의 문제점
구현 방향

단계	상세업무		상세내용	담당자	시작 날짜	마감 날짜	진행도(%)		
분석 계획	프로젝트 계획서 작성		Google Docs로 계획서 작성 및 공유	전원	2025-12-29	2026-01-02	100		
	WBS 작성		Google Sheets로 WBS 작성 및 공유	김훈중	2025-12-29	2026-01-02	100		
설계	ERD설계		ERD Cloud로 ERD 설계 및 테이블 작성	강진용, 이승윤	2025-12-30	2025-12-31	100		
	화면구성도 작성		Canva로 화면 초안 작성	김훈중, 이정륜	2025-12-30	2025-12-31	100		
개발	인프라 구축	Docker 기반 개발·배포 환경 구축	서비스별 Docker Container화 및 docker-compose 기반 통합 오케스트레이션 환경 구성	김훈중	2026-01-02	2026-01-05	100		
		AWS 기반 서버 환경 구축	AWS ECS(Fargate) 기반 서버리스 컨테이너 환경 및 VPC 네트워크 구성	김훈중	2026-01-05	2026-01-06	100		
		CI/CD	GitHub Actions 기반 CI/CD 파이프라인 구축	김훈중	2026-01-07	2026-01-08	100		
		EC2	milvusdb/milvus:v2.3.4 AWS EC2 도커 설치 및 DB구축	이정륜	2026-01-03	2026-01-08	100		
	데이터베이스	PostgreSQL 16-3.4	PostgreSQL 16-3.4 기반 데이터베이스 구축	이승윤	2026-01-02	2026-01-06	100		
		milvusdb/milvus:v2.3.4	milvusdb/milvus:v2.3.4 벡터디비 설치	이정륜, 김훈중	2026-01-02	2026-01-06	100		
	기능 구현	RAG	법령집 문서 RAG (민원 유형/처리기관) 분류 시스템	이정륜	2026-01-05	2026-01-07	100		
		음성 민원 접수(STT)	음성 녹음 기능 구현, STT 변환 정제	이정륜	2026-01-06	2026-01-09	100		
		로그인, 회원가입, 아이디/비번 찾기	회원가입/로그인 구현, 계정 찾기 및 비밀번호 재설정	김훈중	2026-01-09	2026-01-13	100		
		민원접수 지도연결	카카오 지도 API 연동 및 민원 위치 작성	이승윤	2026-01-08	2026-01-08	100		
		텍스트 민원 RAG 연동	텍스트 민원 입력시 RAG 파이프라인 구축	이정륜	2026-01-09	2026-01-09	100		
		이미지 민원 접수(YOLO)	이미지 업로드 기능 구현, YOLO 탐지 API 연동, 민원 유형, 기관 자동 분류	강진용, 이승윤	2026-01-05	2026-01-09	100		
		민원 접수 트랜잭션	민원 기본 정보 저장 후 민원-기관 매핑 데이터를 INSERT로 처리	김훈중	2026-01-14	2026-01-16	100		
		메인페이지	메인 페이지 화면 구현	김훈중	2026-01-10	2026-01-16	100		
		관리자 대시보드	전체 접수 현황 표시, 민원 분류별 차트 및 세부 차트	이정륜	2026-01-10	2026-01-16	100		
		민원 목록 게시판 (사용자, 관리자)	민원 목록 게시판 구현	강진용	2026-01-10	2026-01-16	100		
		GIS(사용자, 관리자)	민원 위치 지도 표시(마커/클러스터), 목록 ↔ 지도 연동, 핫스팟 및 집계 시각화	이승윤	2026-01-10	2026-01-16	100		
		마이페이지	회원 정보 수정 및 탈퇴, 나의 민원 목록 조회	이승윤	2026-01-10	2026-01-16	100		
		통합 테스트		주요 시나리오 기능 테스트 및 오류 수정	전원	2026-01-16	2026-01-20	100	
		최종 발표 자료 작성		PPT 작성 및 발표 준비	전원	2026-01-16	2026-01-23	100	

01

개발 배경

담당 역할

단계별 개발 일정

| **민원 접수의 문제점**

구현 방향

“이 민원은 어느 처리기관일까요?”

심야 시간대 공사 소음으로 수면에
큰 지장이 있습니다.

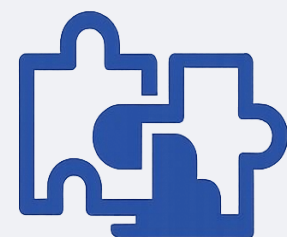
01

개발 배경

담당 역할
단계별 개발 일정

민원 접수의 문제점

구현 방향



접수 창구 파편화

국민신문고 · 안전신문고 · 지자체 사이트로 분산



과도한 유형 선택

신고 전에 정확한 유형 판단 필요



기관 선택 부담

행정 구조를 알아야 접수 가능

“왜 민원 접수는 불편한가?”

안전신고 유형 선택

겨울철 집중신고

도로, 시설물 파손 및 고장

사업장 안전(건설·공사현장 등)

고층 건축물 화재 안전

대기오염

수질오염

소방안전

기타 안전·환경 위험요소

취소

안전신문고

기관분류 ☒ 중앙행정기관 ☐ 지방자치단체 ☐ 교육기관 ☐ 공공기관 ☐ 헌법기관

개인정보보호위원회	경찰청	고용노동부	고위공직자범죄수사처
고준위방사성폐기물관리위원회	공정거래위원회	과학기술정보통신부	관세청
교육부	국가건축정책위원회	국가교육위원회	국가데이터처
국가보훈부	국가유산청	국가인권위원회	국가정보원
국무조정실	국무총리비서실	국민권익위원회	국방부
국세청	국토교통부	금융위원회	기상청
기획예산처	기후에너지환경부	농림축산식품부	농촌진흥청
대검찰청	문화체육관광부	민주평화통일자문회의사무처	방송미디어통신위원회
방위사업청	법무부	법제처	병무청
보건복지부	부마민주항쟁진상규명및 관련자명예회복심의위원회	산림청	산업통상부
새만금개발청	성평등가족부	소방청	식품의약품안전처
외교부	우주항공청	원자력안전위원회	인사혁신처
재외동포청	재정경제부	저출산고령사회위원회	조달청
중소벤처기업부	지식재산처	질병관리청	통일부
해양경찰청	해양수산부	행정안전부	행정중심복합도시건설청

국민신문고

01

개발 배경

담당 역할

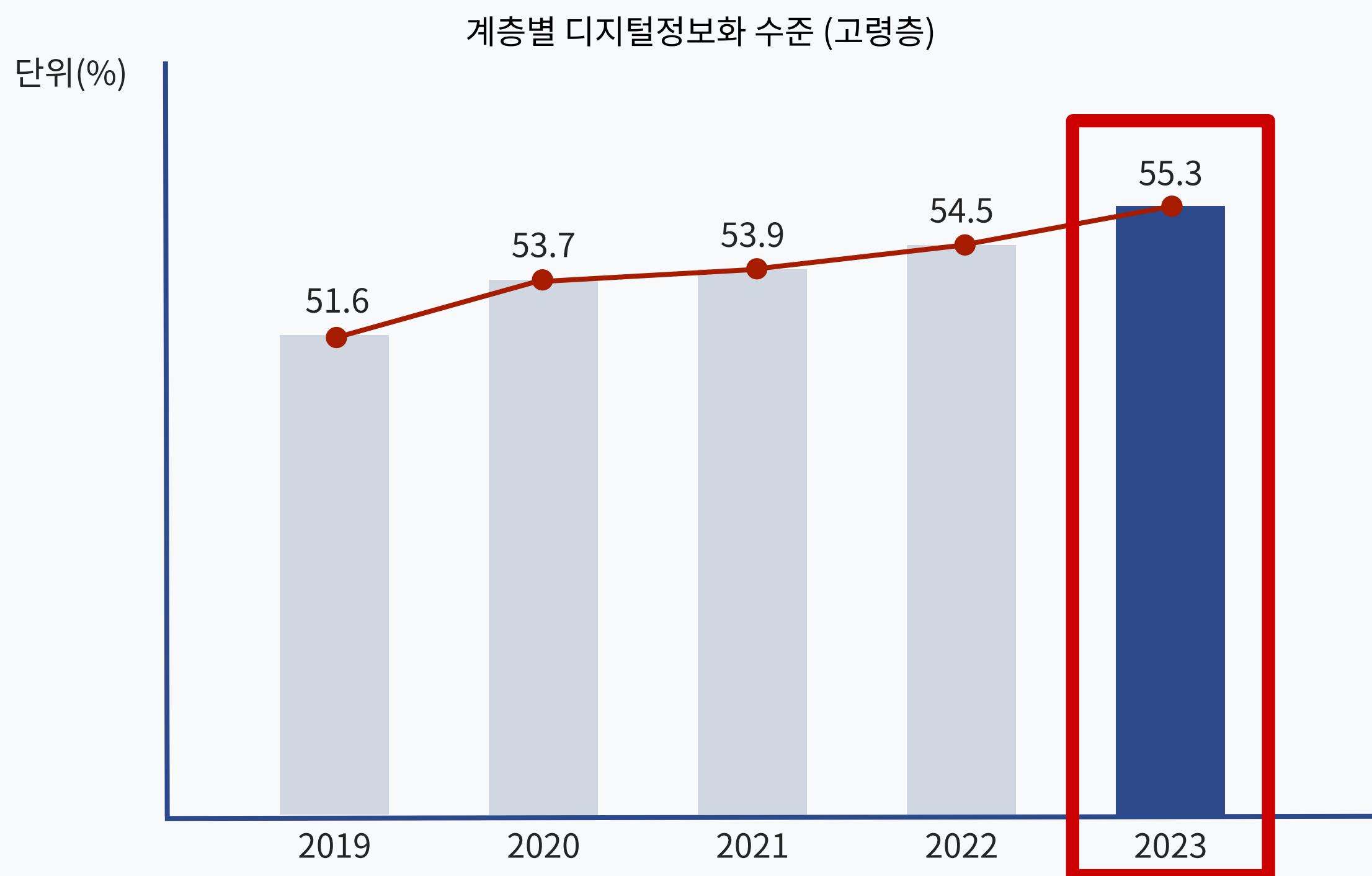
단계별 개발 일정

민원 접수의 문제점

구현 방향

“그 불편함이 누구에게, 얼마나 심각한가?”

과학기술정보통신부의 '2023 디지털정보격차 실태조사'에 따르면, 고령층의 디지털 정보화 수준 역량은 일반 국민 대비 55.3%에 불과 ... 전자정부 서비스로 민원을 해결하는 것이 일상이 된 지금, 많은 노인들이 일상적 불편함 ... 겪고 있습니다.



01

담당 역할
단계별 개발 일정
민원 접수의 문제점
구현 방향

프로젝트 목표

“민원 접수의 판단 부담을 제거하다”



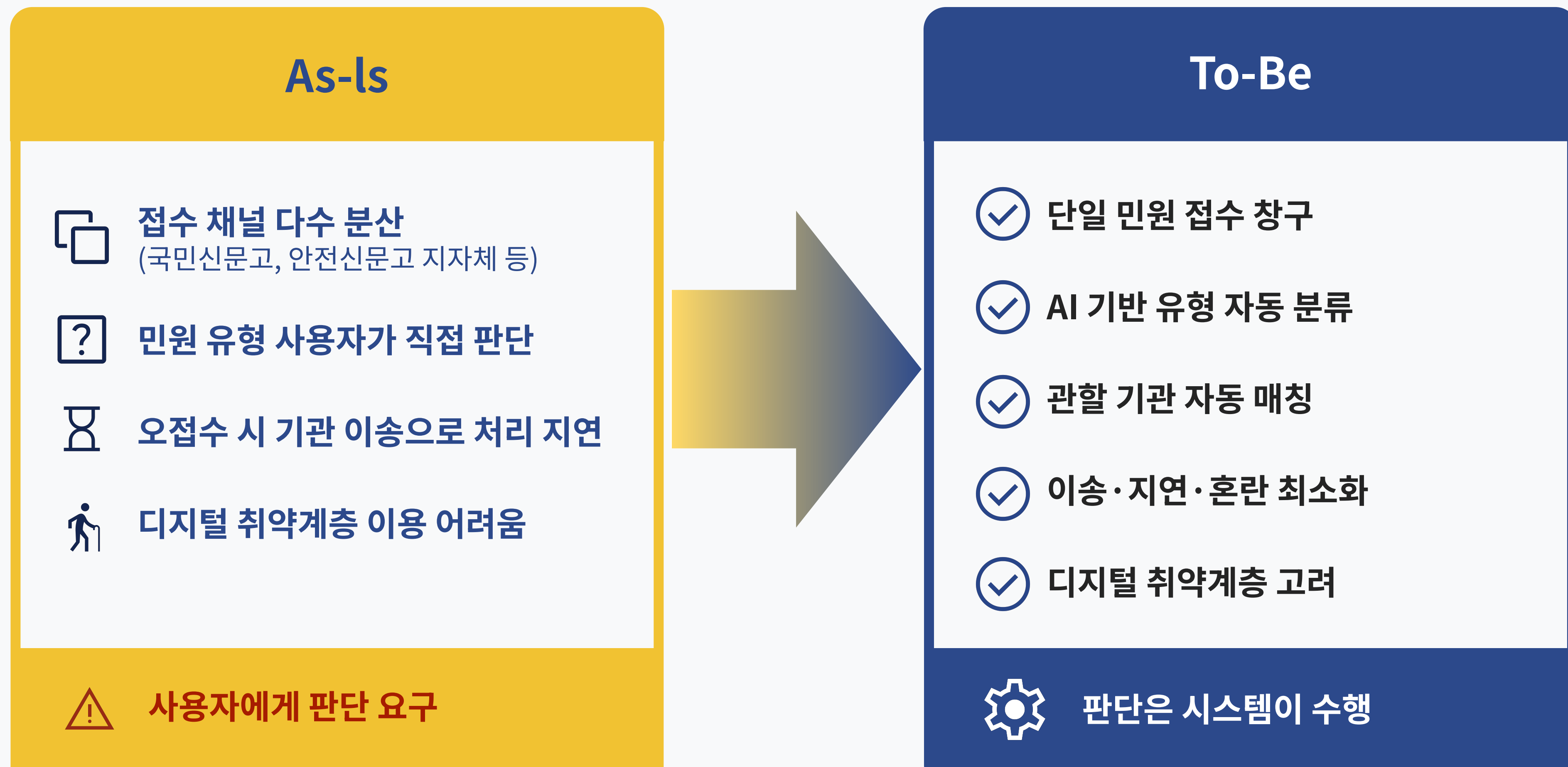
사용자는 판단하지 않는다. 시스템이 대신 판단한다.

01

담당 역할
단계별 개발 일정
민원 접수의 문제점
구현 방향

프로젝트 목표

“기존 시스템과 무엇이 다른가?”



02 프로젝트 및 시스템 설계

02

시스템 아키텍처

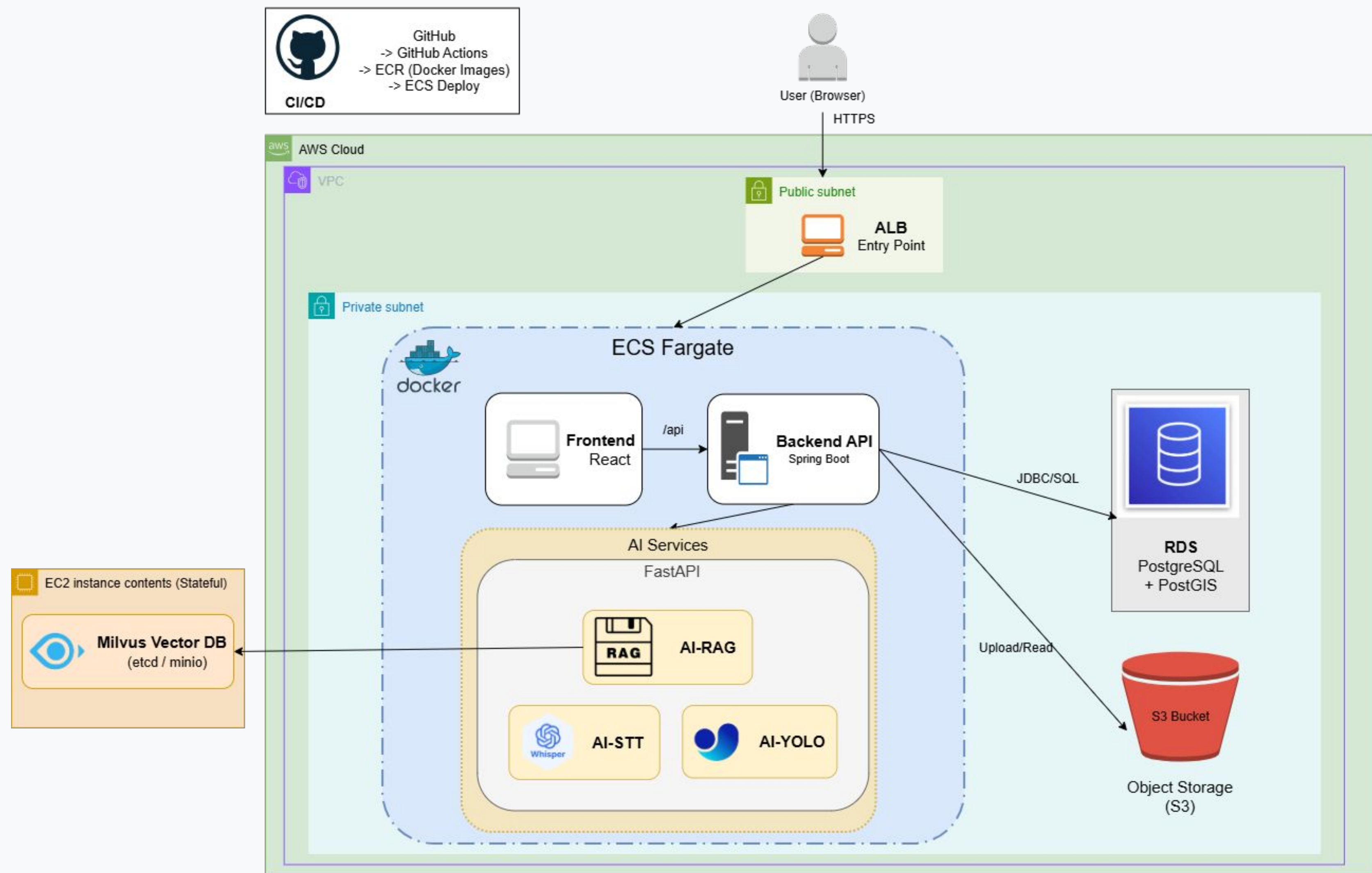
연동 구조

ERD

기술 스택

단계별 처리 과정

분류 기준 설계



02

데이터 모델 설계

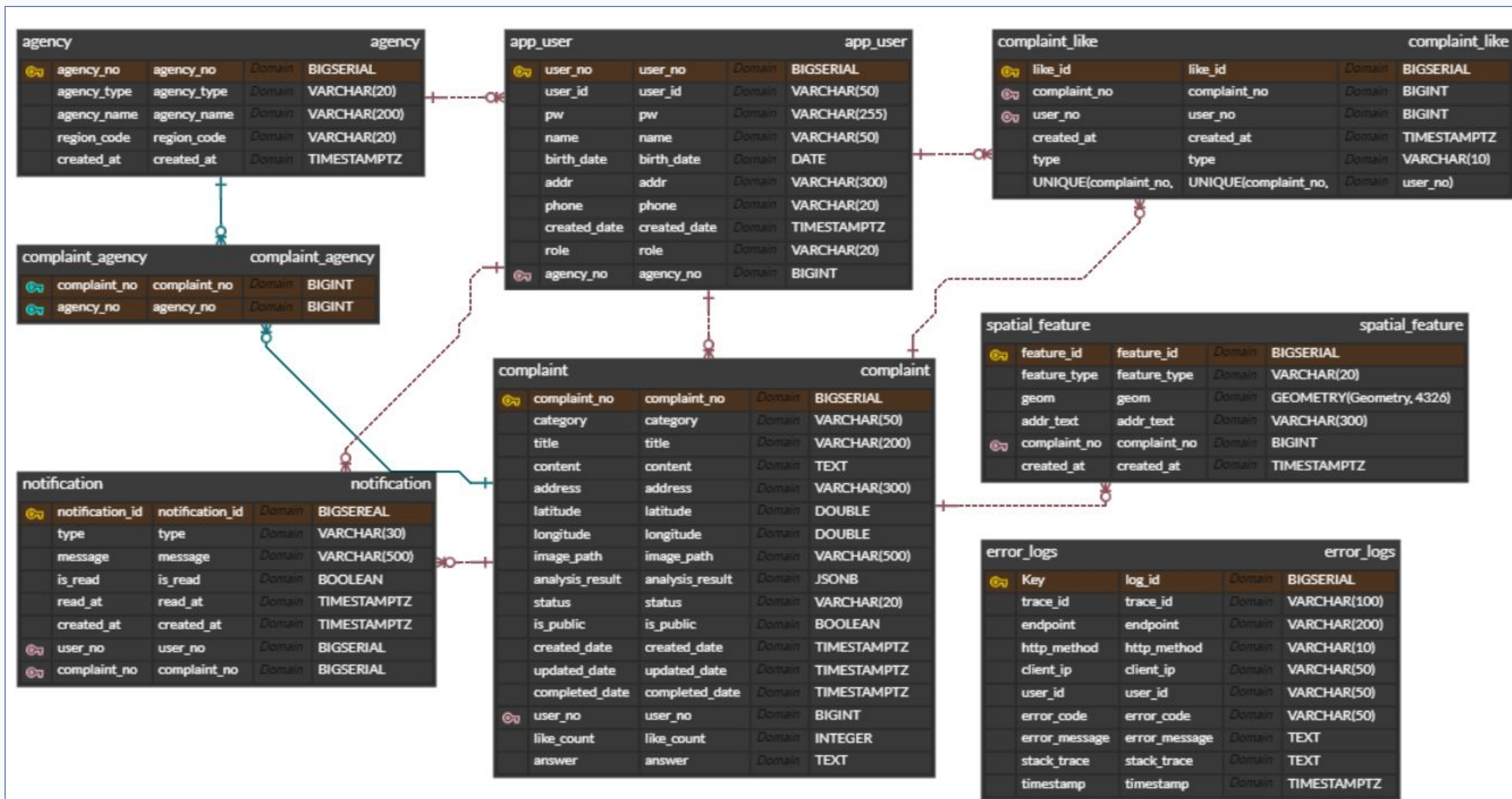
연동 구조

ERD

기술 스택

단계별 처리 과정

분류 기준 설계



02

기술 스택 구성

연동 구조

ERD

기술 스택

단계별 처리 과정

분류 기준 설계

백엔드



Spring



Lombok



Java



Spring Boot



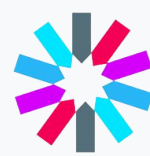
Spring Security



MyBatis



Gradle

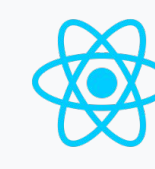


JWT

프론트엔드



JavaScript



React



TypeScript



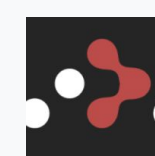
Tailwind CSS



Vite



Node.js

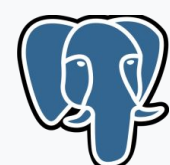


React Router



HTML

데이터베이스



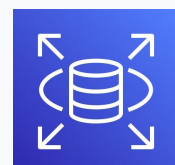
PostgreSQL



Dbeaver



PostGIS



RDS

AI



Python



FastAPI



Ultralytics



OpenAI Whisper



Milvus



PyTorch

인프라



AWS



ECS (Fargate)



ALB



Amazon S3



Docker



GitHub Actions



NGINX

02

연동 구조

ERD

기술 스택

단계별 처리 과정

분류 기준 설계

민원 처리 흐름



02

AI 기반 민원 분류 기준

연동 구조

ERD

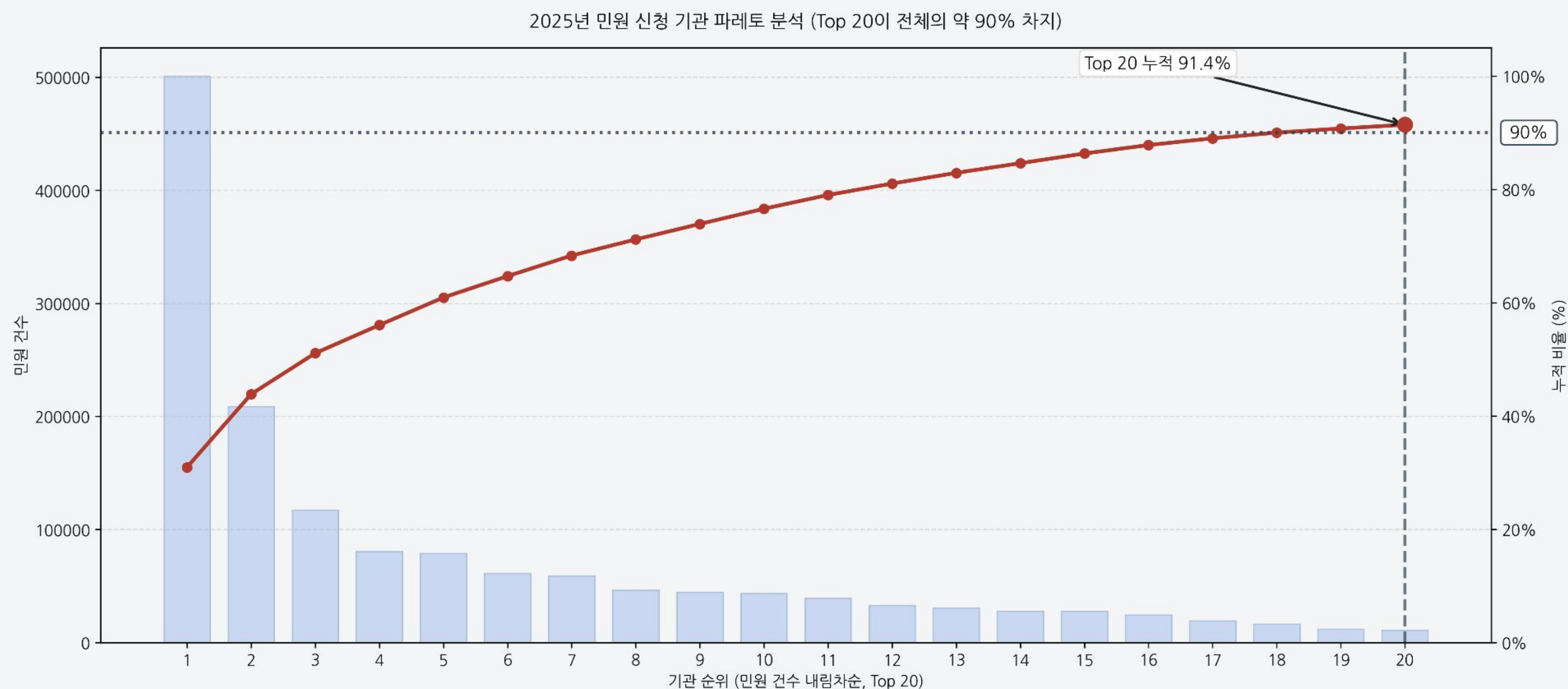
기술 스택

단계별 처리 과정

분류 기준 설계

20개의 핵심 정부 부처와 5대 생활 불편요소를 집중 관리 대상으로 선정

[25.12.30 18:00 기준 상위 20개 기관 / 민원 유형 10개 선택]



1. 경찰청 2. 국토교통부 3. 고용노동부 4. 국방부 5. 국민권익위원회 6. 식품의약품안전처 7. 대검찰청 8. 기획재정부 9. 행정안전부 10. 보건복지부
11. 과학기술정보통신부 12. 국세청 13. 기후에너지환경부 14. 법무부 15. 공정거래위원회 16. 교육부 17. 해양수산부 18. 농림축산식품부 19. 소방청 20. 인사혁신처

03 핵심 기능 및 구현

03

민원 접수 방법

민원 접수

민원 지도 GIS
대시보드



텍스트 민원

글로 민원을 작성하세요



음성 민원

음성으로 민원을 접수하세요



이미지 민원

AI가 사진을 분석합니다

03

민원 접수

민원 지도 GIS
대시보드

텍스트 민원

텍스트로 민원 접수 후 AI가 민원 유형과 담당 기관을 분석

텍스트 민원 신청서

아래 양식을 작성해주세요

민원 제목 *

불법 주정차

민원 내용 *

출퇴근 시간마다 불법 주정차 차량으로 인해 도로 통행이 어렵습니다.
특히 교차로 인근 주정차로 사고 위험이 높아 단속을 요청드립니다.

74 / 1,000자

 AI 분석 결과

 민원 유형

경찰·검찰

 처리 기관

경찰청

 AI 분석하기

유사 민원·법령 데이터 기반 분석

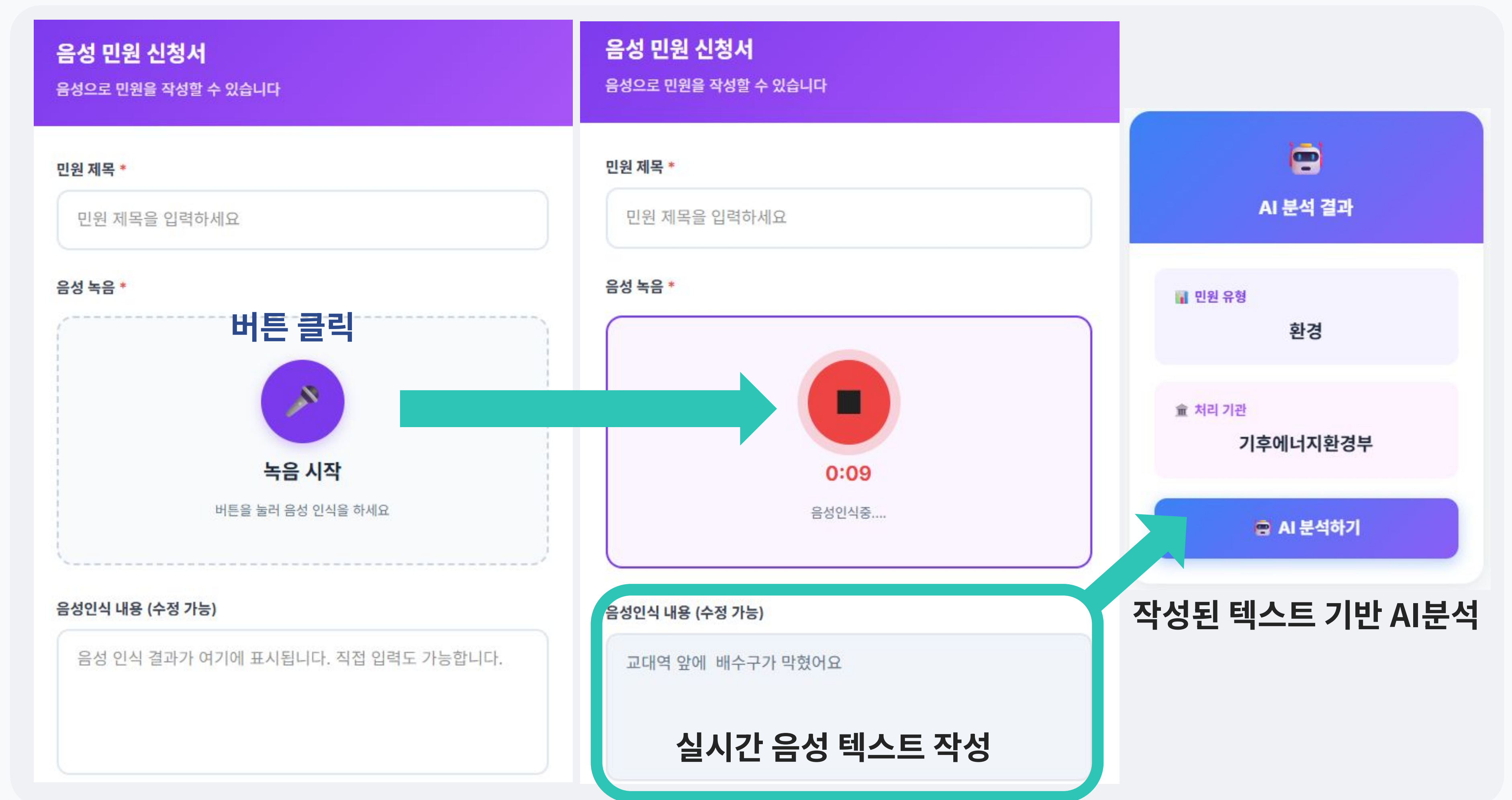
03

음성 민원

민원 접수

민원 지도 GIS
대시보드

음성 인식 후 AI가 민원 유형과 담당 기관을 분석



03

민원 접수

민원 지도 GIS
대시보드

이미지 민원

이미지 인식 후 AI가 민원 유형과 담당 기관을 분석

이미지 민원 신청서

사진을 업로드하면 AI가 분석합니다

민원 제목 *

민원 제목을 입력하세요

현장 사진 첨부 * (5MB 이하의 이미지 파일만 업로드 가능합니다)

버튼 클릭 후 사진 업로드



클릭하여 사진을 업로드하세요

민원 내용 (자동 생성됨)

사진을 업로드하면 AI가 분석 결과를 입력합니다

이미지 민원 신청서

사진을 업로드하면 AI가 분석합니다

민원 제목 *

민원 제목을 입력하세요

현장 사진 첨부 * (5MB 이하의 이미지 파일만 업로드 가능합니다)



민원 내용 (자동 생성됨)

[AI 이미지 분석 결과]

유형: 교통

담당: 경찰청



AI 분석 결과

민원 유형

교통

처리 기관

경찰청

사진에서 자동으로 정보를 추출합니다

업로드된 이미지 기반 AI분석

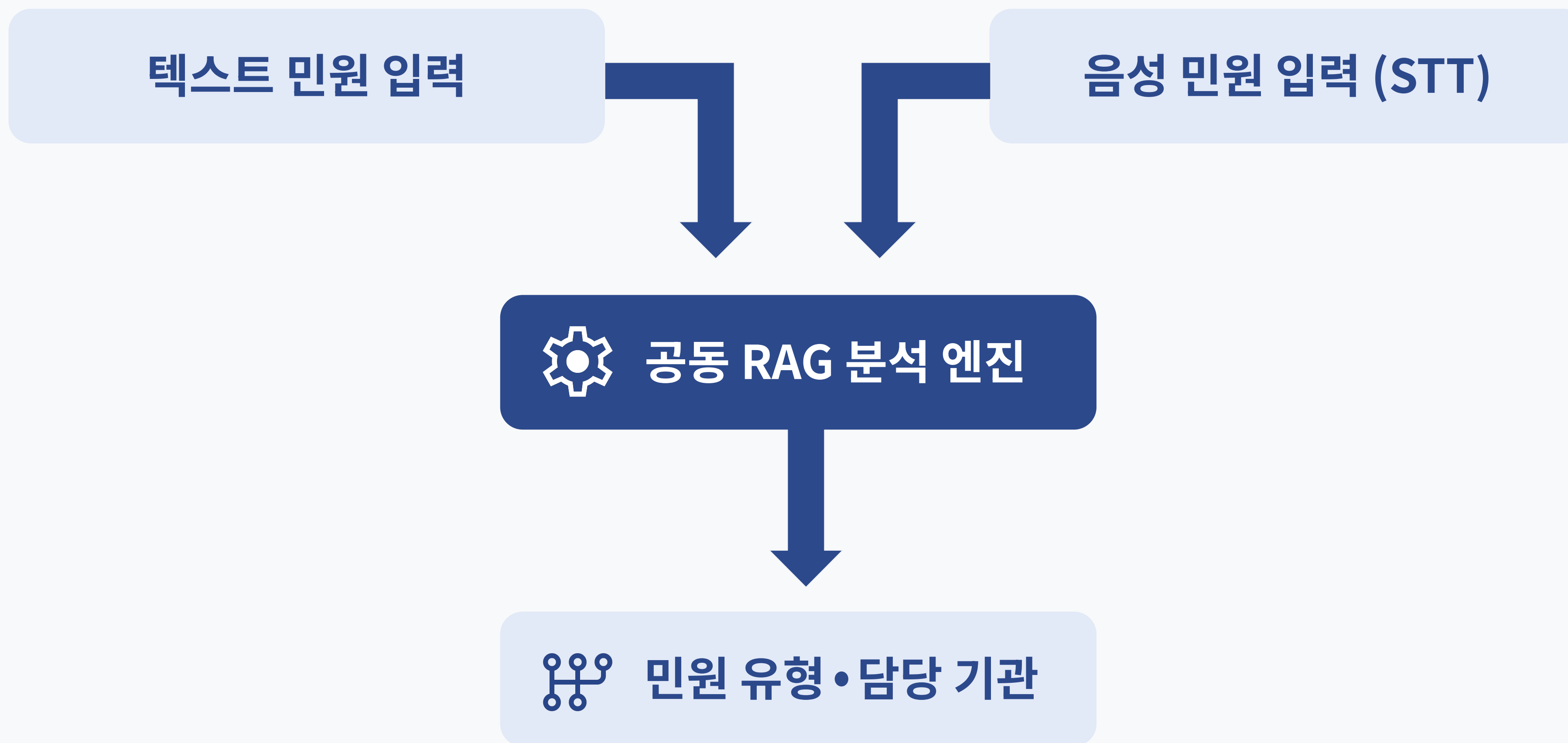
03

민원 접수

민원 지도 GIS

대시보드

텍스트· 음성 민원 처리 구조

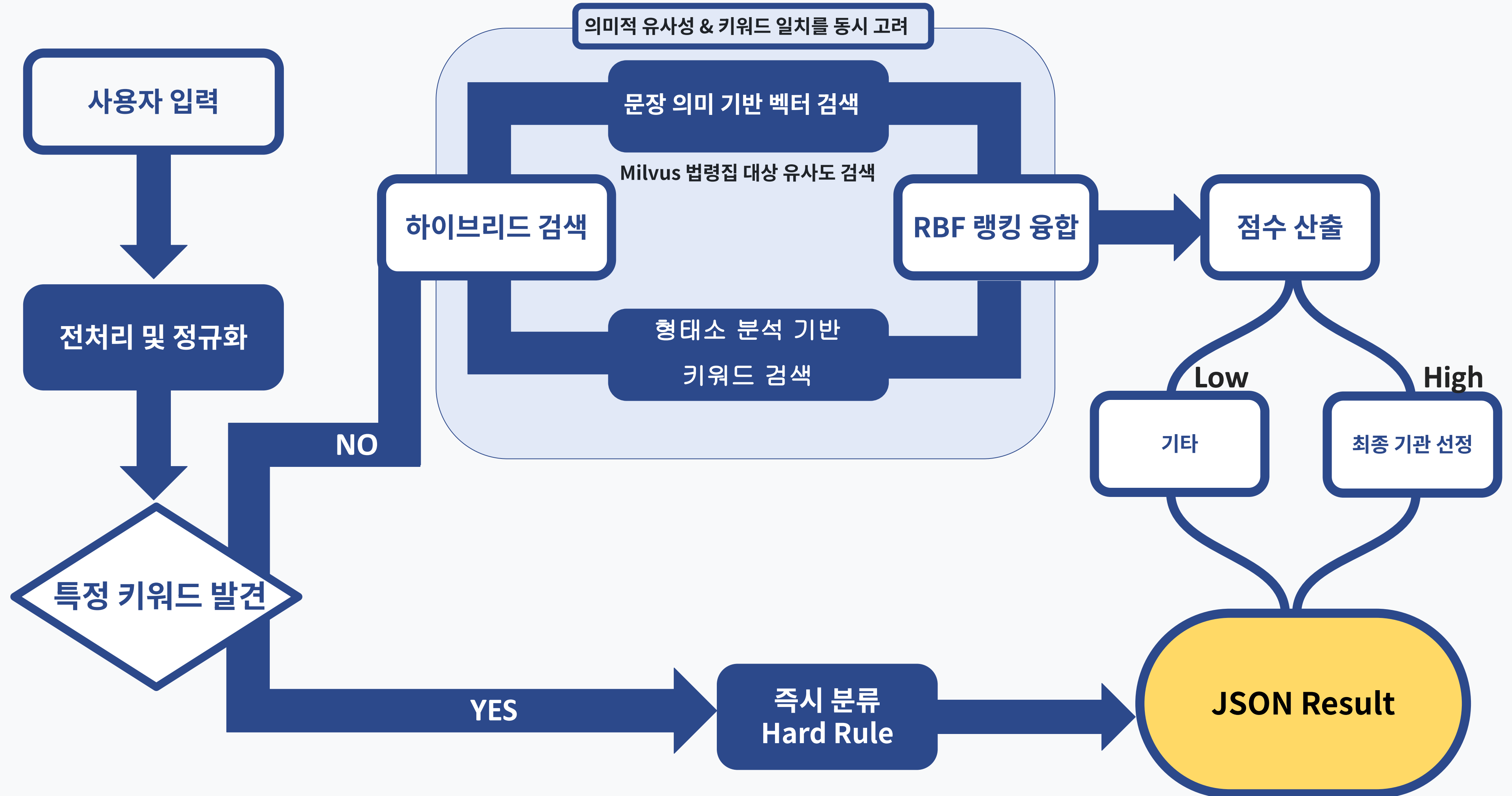


03

RAG 민원 처리 방법

민원 접수

민원 지도 GIS
대시보드

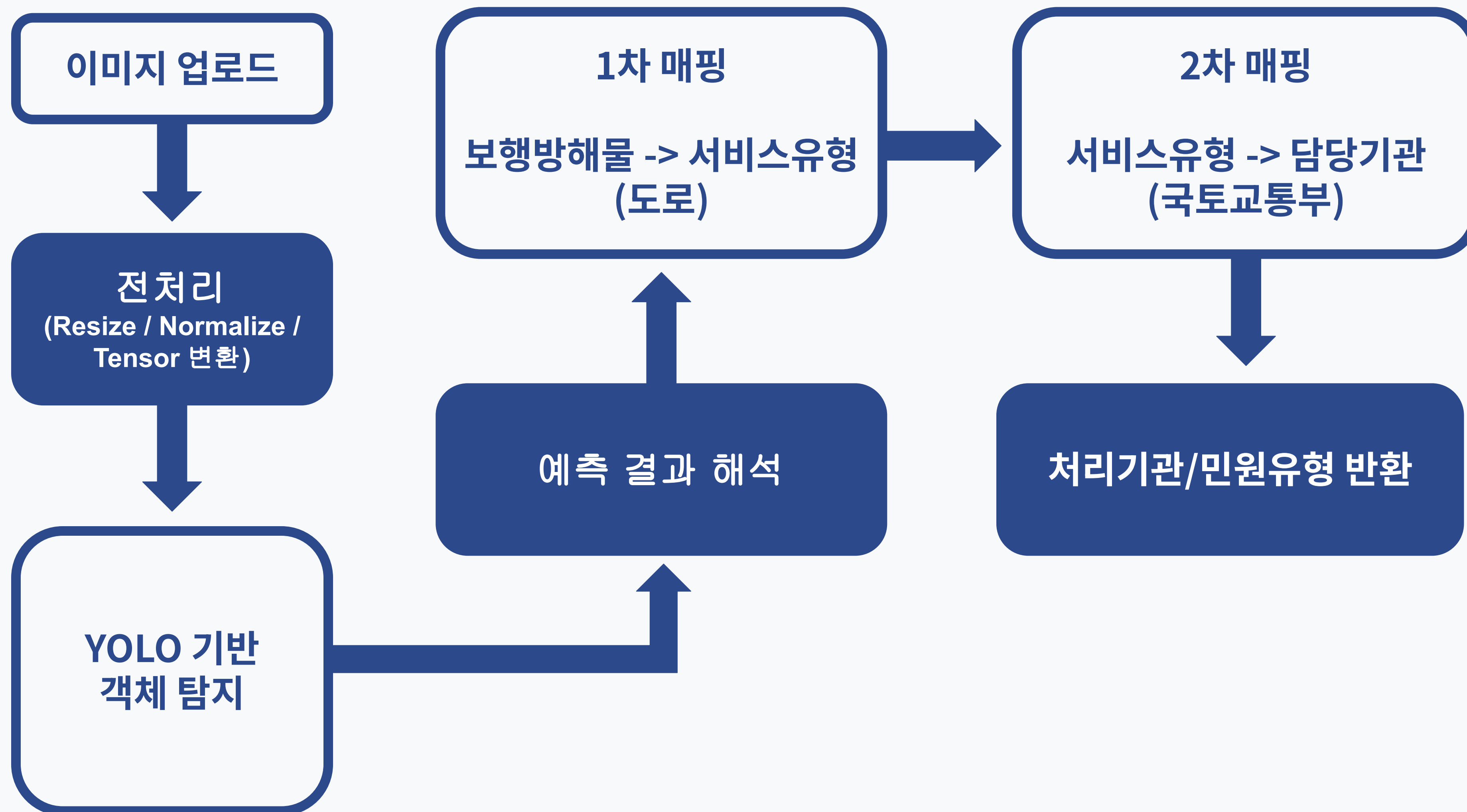


03

YOLO 분석

민원 접수

민원 지도 GIS
대시보드



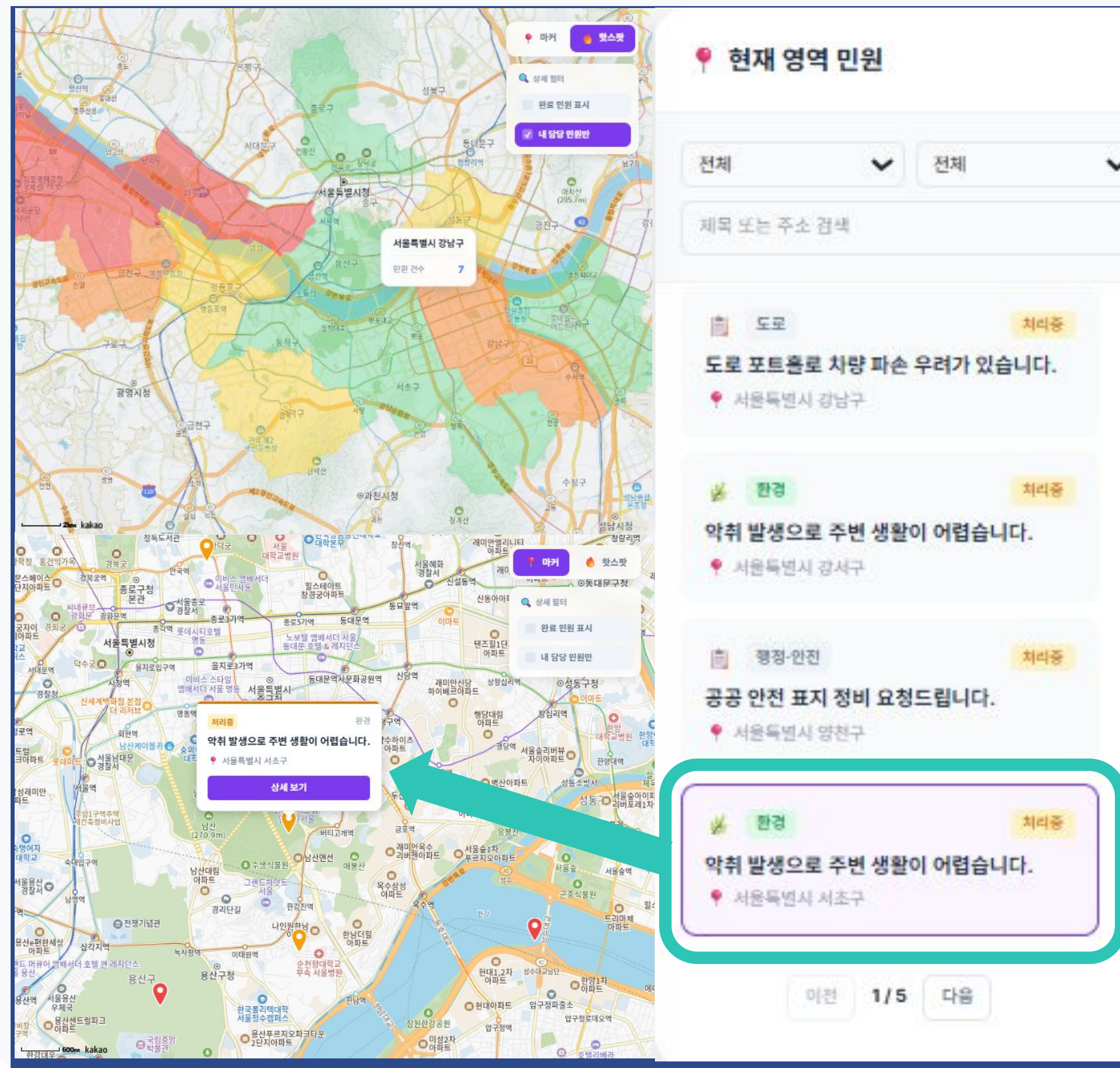
03

민원 지도 GIS

민원 접수

민원 지도 GIS

대시보드



03

담당자 대시보드

민원 접수
민원 지도 GIS

대시보드

1. 실시간 민원 현황 모니터링

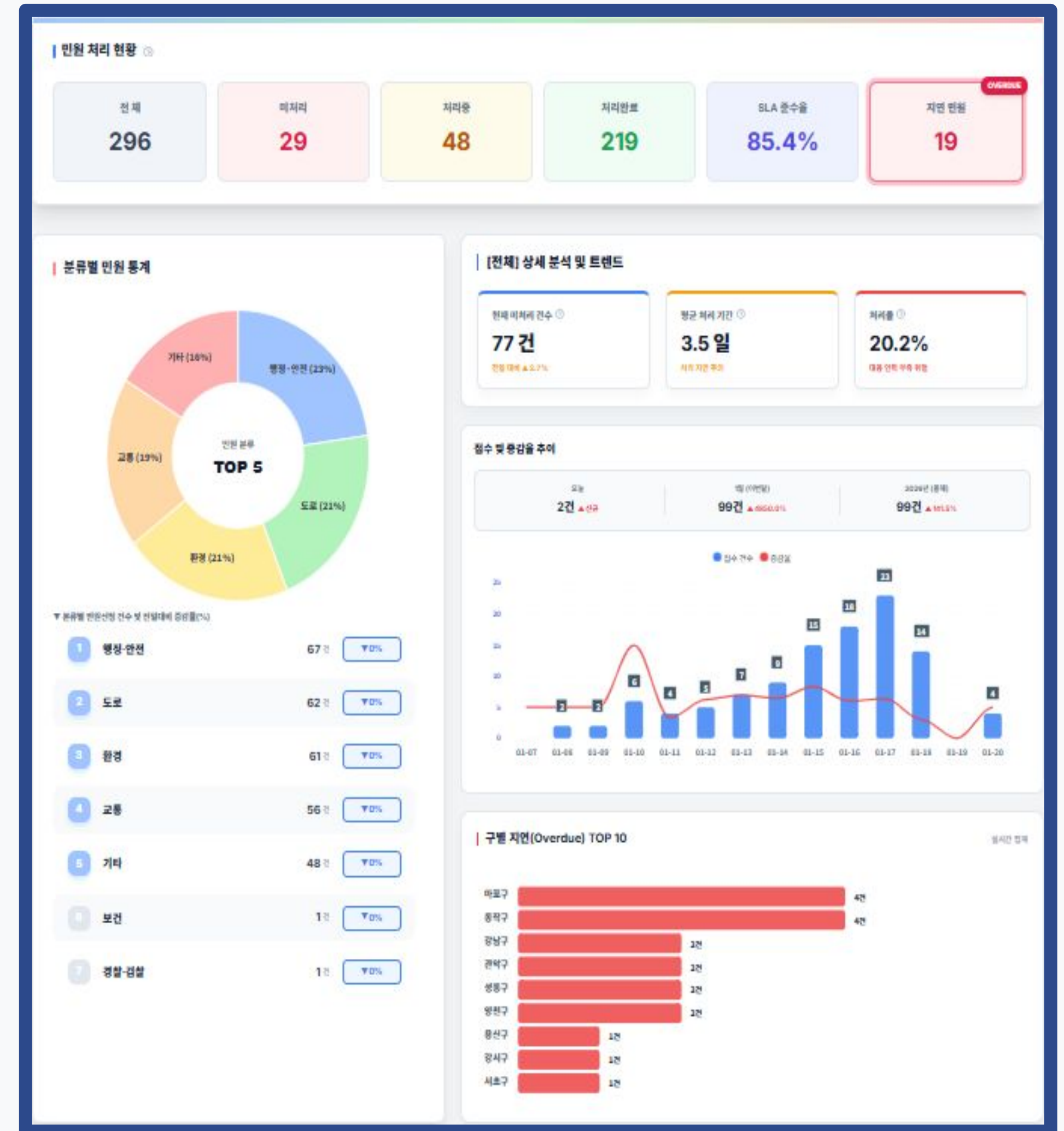
- 30초마다 갱신
- 민원 처리 현황 실시간 집계
- SLA(서비스 수준 협약) 준수율 및 평균 처리 기간 자동 산출

2. 데이터 시각화

- **분야별 통계:** 불법주정차, 도로, 안전 등 민원 유형별 발생 빈도 분석
- **기간별 추이:** 일별/월별/연별 접수 및 처리량 변화 그래프 제공

3. 지연 처리 방지

- 장기 미처리 민원 자동 알림



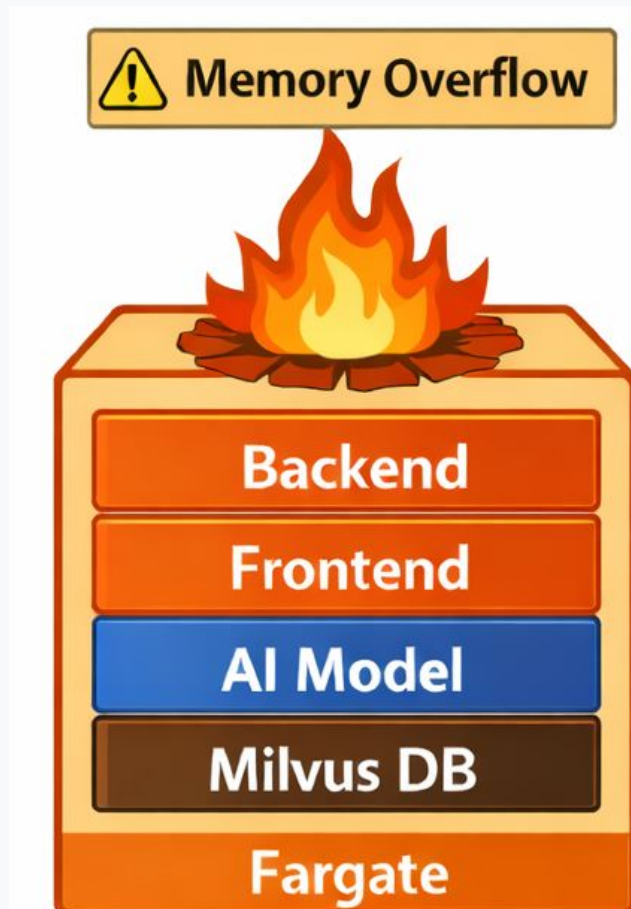
04 문제 해결 방법 및 협업 관리

04

Milvus DB 안정화를 위한 인프라 분리

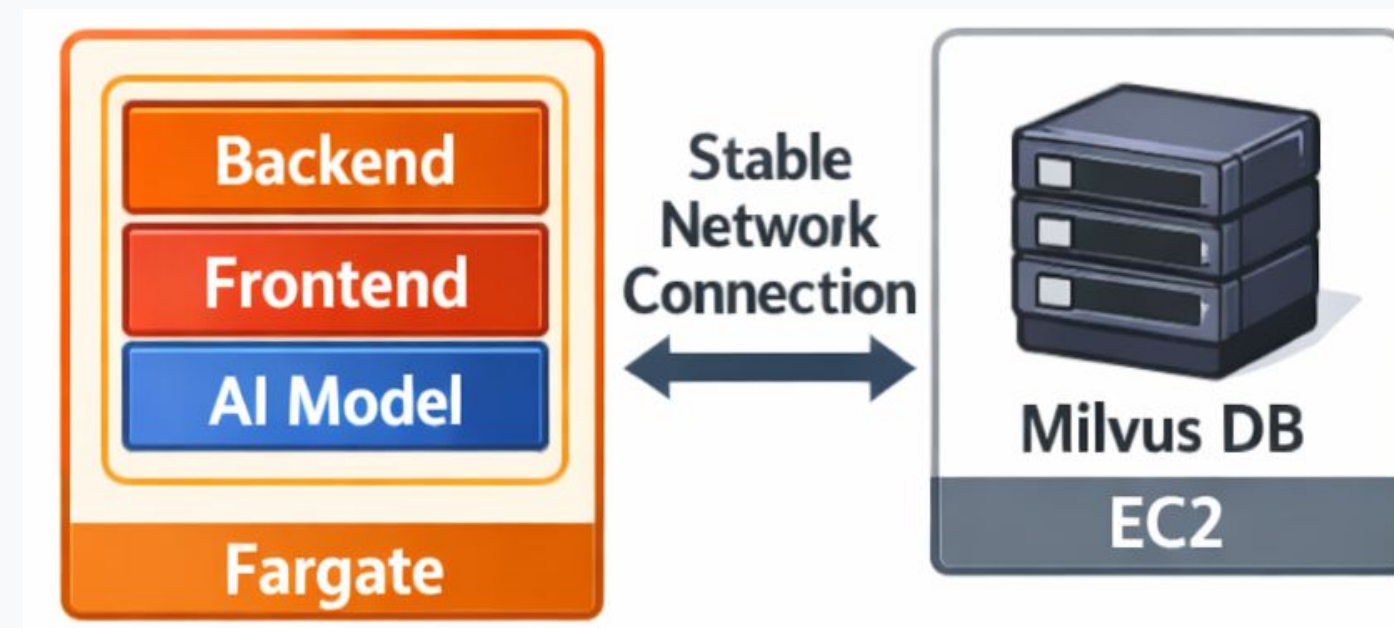
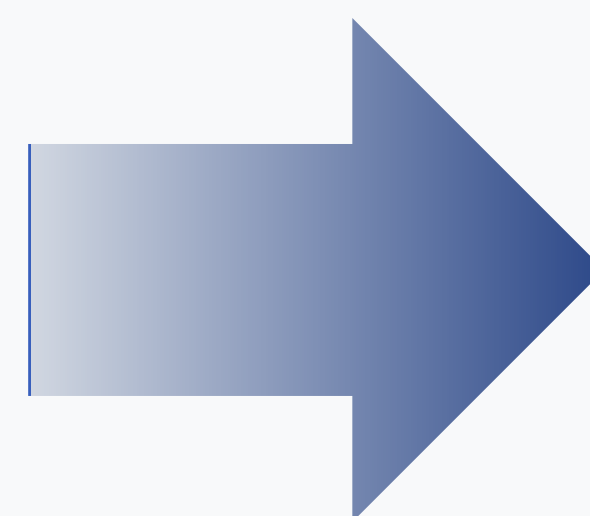
서버 문제 해결

협업 관리



문제 상황

Milvus DB 적재시 다른 컨테이너까지
메모리 부족으로 강제 종료



해결 방법

Milvus 전용 EC2를 할당하여
안정적인 메모리·CPU 자원 보장

04

GitHub 기반 협업 프로세스 구축

서버 문제 해결

협업 관리



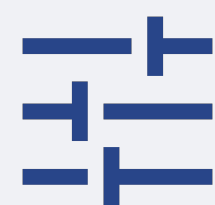
Wiki기반 문서화

프로젝트 구조 실행 배포 절차 체계화
환경 오류 및 실행 방법을 문서로 공유



Issue · Milestone 관리

버그 및 개선 사항을 이슈 단위로 관리
마일스톤을 통해 기능별 진행 상황 가시화



Template 기반 협업 표준화

PR/ISSUE 템플릿을 만들어 협업 품질 향상

000. 프로젝트 실행 및 배포 가이드

jeongryuni edited this page 3 days ago · [1 revision](#)

프로젝트 실행 및 배포 가이드

이 문서는 SafeGuard 프로젝트를 로컬에서 개발하는 방법과 서버에 배포하는 방법을 안내합니다.

1. 로컬 개발 환경 (Local Development)

개발 속도를 높이고 디버깅을 쉽게 하기 위해 백엔드와 프론트엔드를 각각 실행하는 방법을 권장합니다.

필수 준비물

- Java 17 (JDK)
- Node.js 18+
- Docker & Docker Compose (DB 및 AI 서비스 실행용)

1단계: 인프라 실행 (DB, AI 서비스)

백엔드가 의존하는 데이터베이스와 AI 서비스를 먼저 Docker로 실행합니다.

```
# 프로젝트 루트 폴더에서  
docker-compose up -d db ai-yolo ai-stt
```

⚠️ 중요: 도커 컨테이너 상태 관리

- 개발할 때: `safeguard-backend` 와 `safeguard-frontend` 컨테이너는 꺼져 있어야 (Grey/Stop) 합니다. 켜져 있으면 포트(8080, 80)가 충돌해서 우리가 실행하는 코드가 에러납니다.
- 배포/테스트할 때: 모든 컨테이너를 켜야(Blue/Start) 합니다.

2단계: 백엔드 실행 (Spring Boot)

1. IntelliJ 등 IDE에서 프로젝트 열기 (`c:\project_SafeGuard\backend`).
2. `src/main/resources/application.properties` 확인 (DB 설정 등).
3. `SafeGuardApplication.java` 파일을 열고 **Run** (▶) 버튼 클릭.
 - 또는 터미널에서: `./gradlew bootRun` (Windows는 `gradlew.bat bootRun`)
4. 서버 시작 확인: "Tomcat started on port(s): 8080" 로그 확인.

3단계: 프론트엔드 실행 (React)

1. 프로젝트 루트 폴더에서 `npm install` 실행 (필수 패키지 설치)
2. `src/main/resources/application.properties` 확인 (DB 설정 등)
3. `SafeGuardApplication.java` 파일을 열고 **Run** (▶) 버튼 클릭
◦ 또는 터미널에서: `./gradlew bootRun` (Windows는 `gradlew.bat bootRun`)
4. 서버 시작 확인: "Tomcat started on port(s): 8080" 로그 확인
5. 프론트엔드 실행: `npm start` 실행 (브라우저에서 `localhost:3000` 접속)



05

결론 및 시연

05

기대 효과

기대 효과

시연

- 분산된 민원 접수 채널 단일화
- 민원 유형 및 기관 분류 자동화 구현
- 민원 처리 및 운영 계획 수립 효율화
- 위험 지역(Hotspot) 시각화를 통한 실시간 사고 예방

기대 효과

| 시연

모두의 민원
by SafeGuard